Java Full Stack Development

2. Spring Security

Module Topics

- Spring Security Motivation
- Architecture Overview
- The Filter Chain
- Method Security
- Testing

Spring Security Motivation

© 2026 by Innovation In Software Corporation

The Security Problem

- Security
  - Property of the entire system
    - A payment service can have excellent business logic and still be catastrophically insecure
  - Security is not localized
    - A single unprotected endpoint can compromise the entire application
  - Security is adversarial
    - Unlike performance or reliability, someone is actively trying to break it

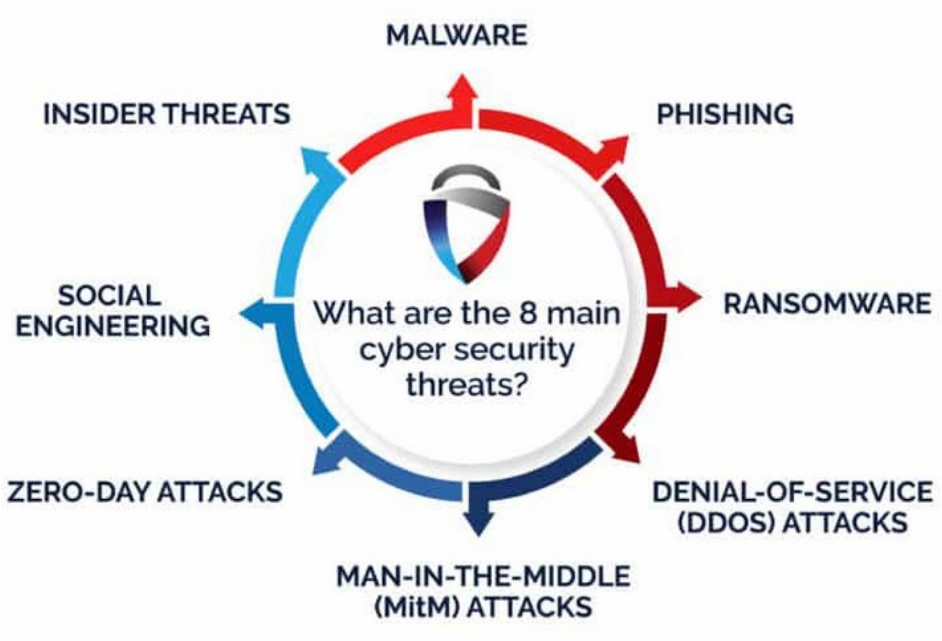


Image Credit: https://fortifydata.com/blog/what-are-the-5-main-threats-to-cybersecurity/

© 2026 by Innovation In Software Corporation

The Security Problem

- Security
  - Security failures are often silent
    - You may not know you have been breached until long after the fact
  - Security requirements are always on
    - There is no "security mode" but it has to work for every request, every time

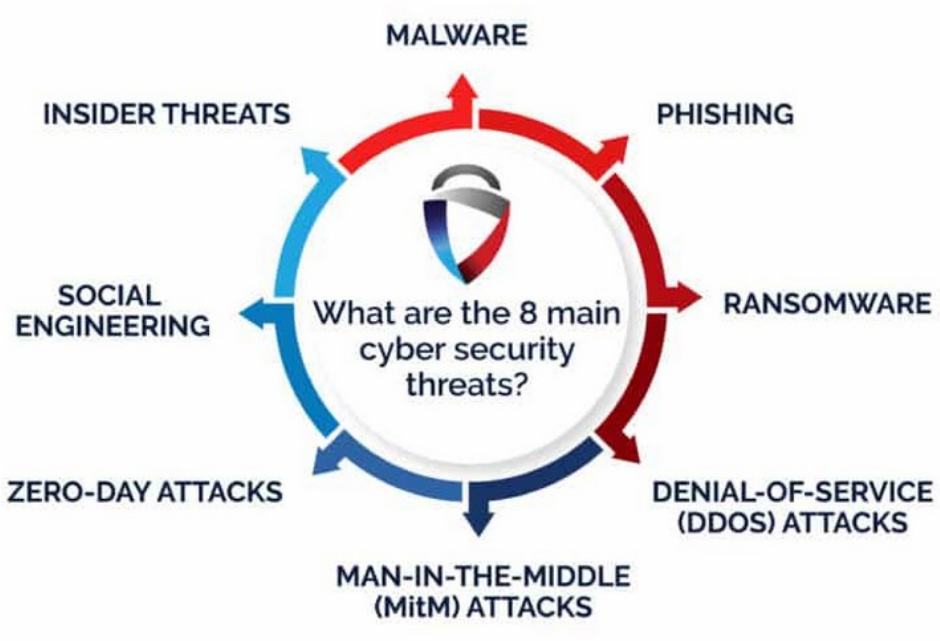
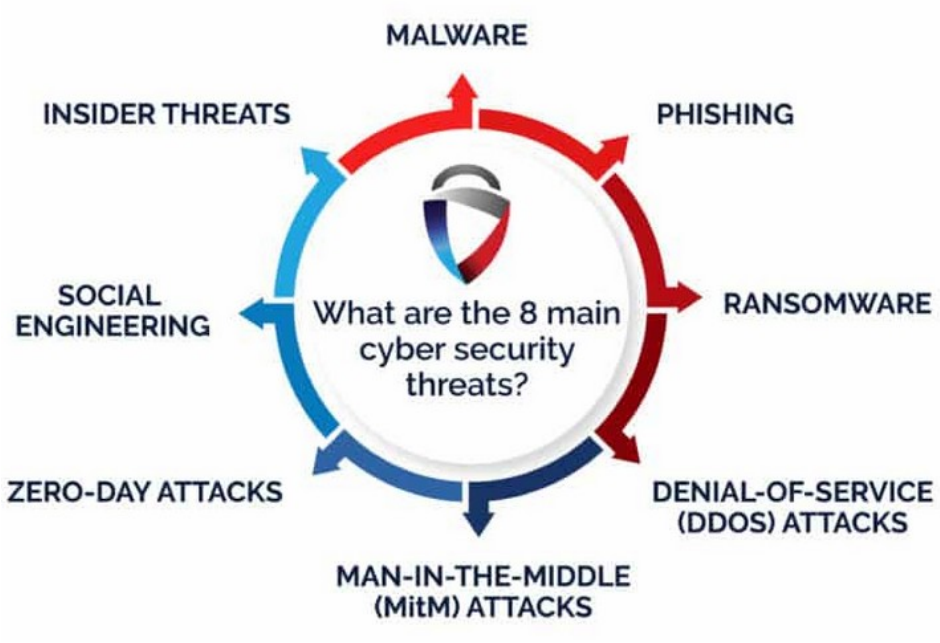


Image Credit: https://fortifydata.com/blog/what-are-the-5-main-threats-to-cybersecurity/

© 2026 by Innovation In Software Corporation

The Security Problem

- To be resolved on every request
  - Authentication
    - Is this caller who they claim to be?
  - Authorization
    - Is this caller permitted to perform this specific action?

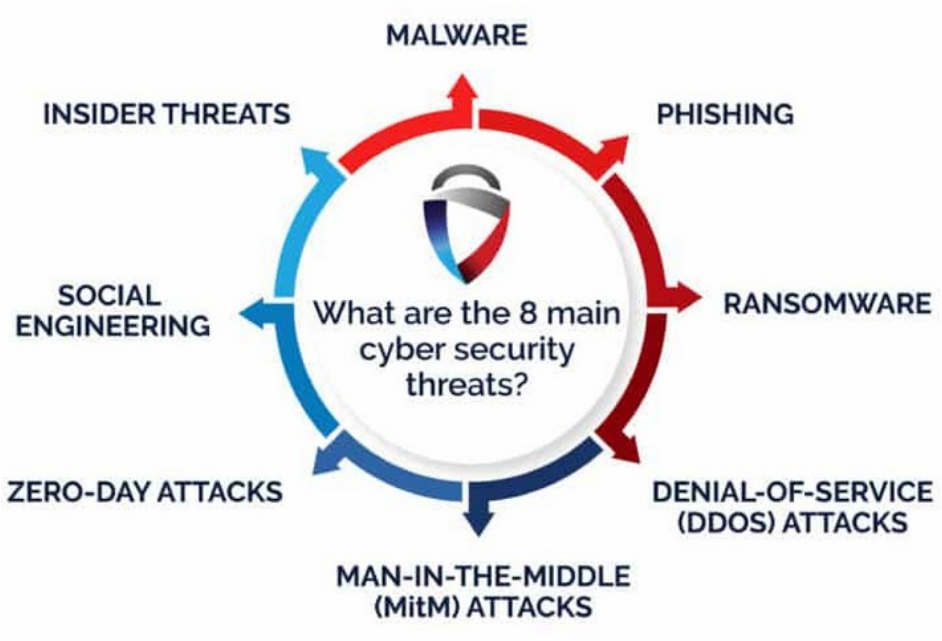


© 2026 by Innovation In Software Corporation

Image Credit: https://fortifydata.com/blog/what-are-the-5-main-threats-to-cybersecurity/

The Security Problem

- To be resolved on every request
  - Data integrity
    - Has this request been tampered with in transit?
  - Confidentiality
    - Is sensitive data protected from exposure?
  - Auditability
    - Can we reconstruct what happened, and can we identify who did it?

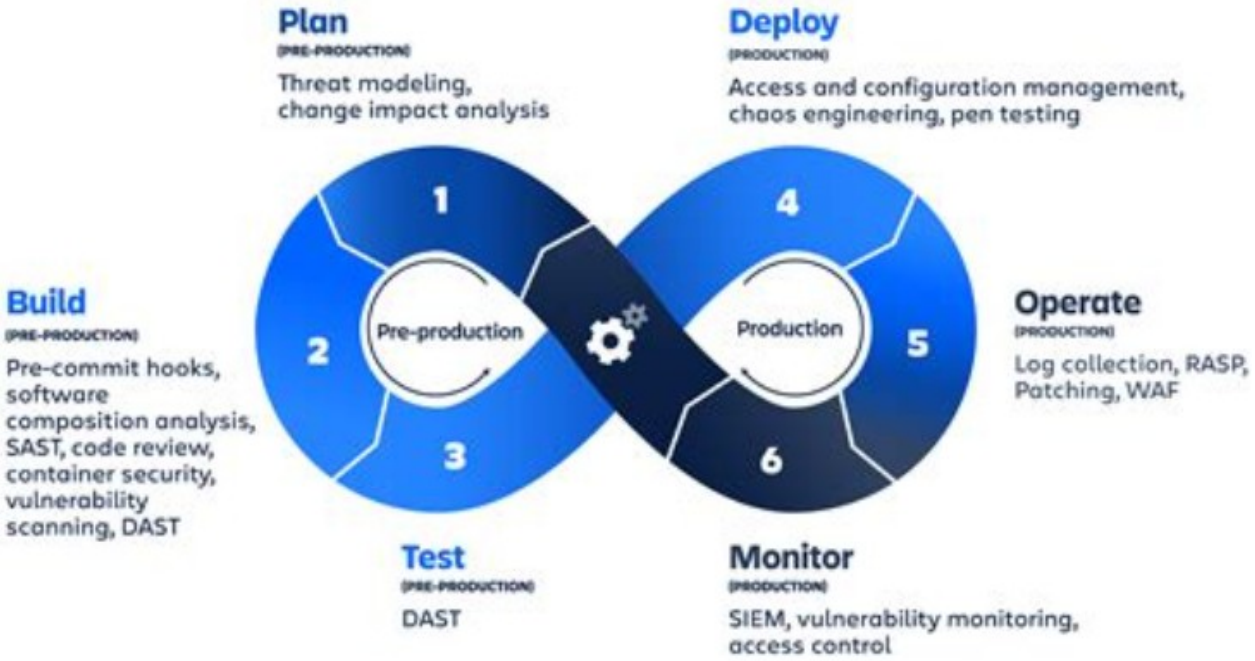


© 2026 by Innovation In Software Corporation

Image Credit: https://fortifydata.com/blog/what-are-the-5-main-threats-to-cybersecurity/

The Security Problem

- The adversarial nature of security
  - For most functionality, like searching
    - The failure mode is a bad result
    - But it is disruptive and not necessarily catastrophic
  - But for authentication
    - The failure mode is an attacker gaining unauthorized access
    - This is potentially irreversible
  - Demands a different engineering posture
    - Not "does this work for valid input?"
    - But "what happens with malicious input?"

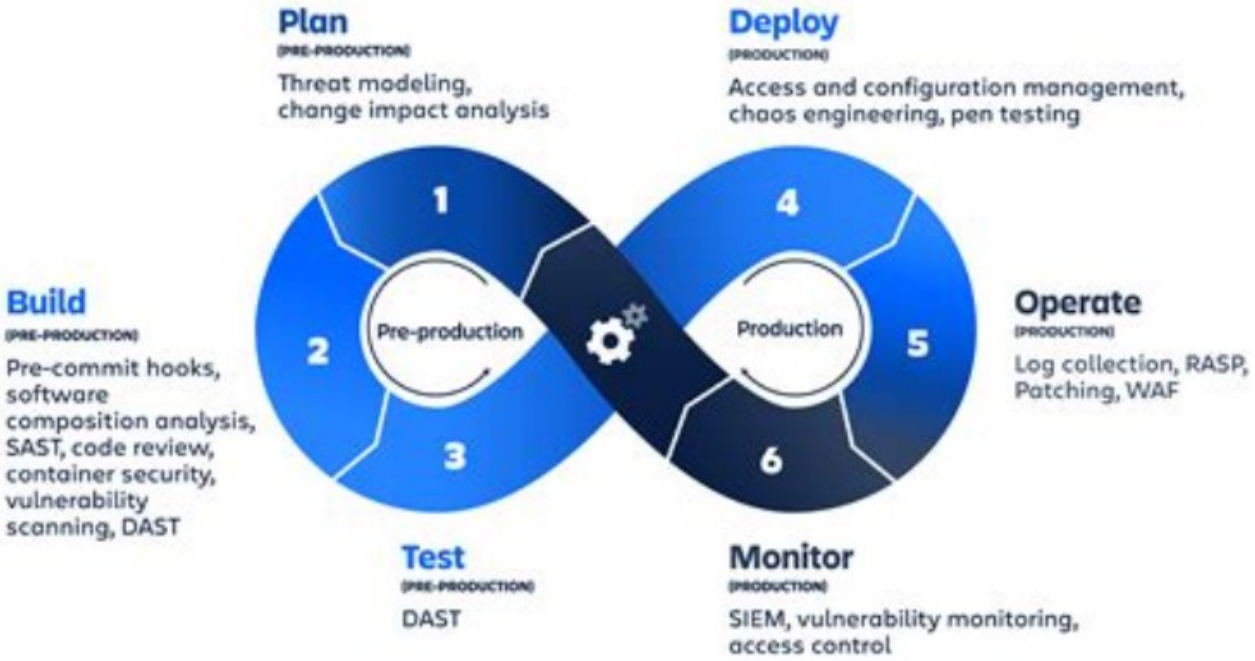


© 2026 by Innovation In Software Corporation

Image Credit: https://imzye.com/DevSecOps/

The Security Problem

- "Always on"
  - Security is a constraint on system design
  - Adding it retroactively is far more expensive and error-prone than designing for it from the start
- DevSecOps
  - Security is foundational at ever step of development and operations
  - Security is a fundamental design building block



© 2026 by Innovation In Software Corporation

Image Credit: https://imzye.com/DevSecOps/

REST Attack Surface

- BOLA example
  - Broken Object Level Authorization
  - The pattern
    - GET /api/accounts/12345 works for the account owner
    - Does it also work when a different authenticated user replaces 12345 with 12346?
  - Yes if the authorization check is only
    - “Is the user authenticated?"
  - And not
    - "is the user authorized to access this specific resource?"

```
Internet
   |
   v
+-------------------------------------------------------------+
|                    Your REST API                            |
|                                                             |
|  POST /api/users          <- Who can create users?          |
|  GET  /api/accounts/{id}  <- Can any user see any acct?      |
|  PUT  /api/transfer       <- Is the caller authenticated?   |
|  GET  /api/admin/reports  <- Is admin access controlled?     |
|  DELETE /api/records/{id} <- Can records be deleted by       |
|                              anyone who guesses the ID?     |
+-------------------------------------------------------------+
```

© 2026 by Innovation In Software Corporation

Common REST Attacks

Attack	Description	Consequence
Broken Authentication	No or weak credential validation	Account takeover
Broken Authorization	User A accesses User B's resources	Data breach
Privilege Escalation	Regular user performs admin actions	System compromise
Token Theft/Replay	Stolen token used to impersonate user	Full account access
Insecure Direct Object Reference	Guessing resource IDs to access others' data	Data exposure
Injection	Malicious input executed by the system	Data destruction

© 2026 by Innovation In Software Corporation

The Naive Approach

- Security in application code
- Problems with this approach
  - Security logic is scattered across every controller
  - Logic is duplicated and diverges over time
  - One forgotten check leaves an endpoint unprotected
  - Custom crypto/validation code is high-risk
  - Inconsistent error responses confuse API consumers

```java
// A controller method with inline, ad-hoc security logic
@GetMapping("/api/accounts/{accountId}")
public Account getAccount(@PathVariable String accountId,
                          HttpServletRequest request) {

    // Step 1: Extract and parse the token manually
    String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        throw new RuntimeException("No token");   // What status code? What for
    }
    String token = authHeader.substring(7);

    // Step 2: Validate the token — but how? Wrote this myself?
    String userId = myCustomTokenValidator.validate(token); // Hope this is cor

    // Step 3: Check permission — but this logic is duplicated in 30 other cont
    if (!userService.hasPermission(userId, "accounts:read")) {
        throw new RuntimeException("Forbidden");  // Again — what format? What
    }

    // Step 4: Finally, business logic
    return accountService.findById(accountId);
}
```

© 2026 by Innovation In Software Corporation

The Naive Approach

- The problems emerge at scale:
  - With 30 controllers
    - Token extraction and validation code lives in 30 places
    - When the validation logic needs to change then 30 files need to be updated
  - The RuntimeException approach
    - Gives no guarantees about HTTP status codes or response format
    - One controller might return a 500, another a 400, another no body at all
  - "Hope this is correct"
    - Custom cryptographic validation of tokens is almost never correct.
    - The JWT specification covers cases that custom implementations routinely get wrong

```java
// A controller method with inline, ad-hoc security logic
@GetMapping("/api/accounts/{accountId}")
public Account getAccount(@PathVariable String accountId,
                          HttpServletRequest request) {

    // Step 1: Extract and parse the token manually
    String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        throw new RuntimeException("No token");   // What status code? What for
    }
    String token = authHeader.substring(7);

    // Step 2: Validate the token — but how? Wrote this myself?
    String userId = myCustomTokenValidator.validate(token); // Hope this is cor

    // Step 3: Check permission — but this logic is duplicated in 30 other cont
    if (!userService.hasPermission(userId, "accounts:read")) {
        throw new RuntimeException("Forbidden");  // Again — what format? What
    }

    // Step 4: Finally, business logic
    return accountService.findById(accountId);
}
```

© 2026 by Innovation In Software Corporation

The Naive Approach

- Problems with the manual approach
  - Scattered enforcement
    - Security logic lives in every controller method
  - Easy to forget
    - There is nothing forcing a developer to add it
  - No single place to audit
    - You must check every endpoint individually
  - Inconsistent implementation
    - Different developers implement the same check differently
    - Edge cases handled differently across the codebase
    - Refactoring is dangerous because changes must be made everywhere

```java
// A controller method with inline, ad-hoc security logic
@GetMapping("/api/accounts/{accountId}")
public Account getAccount(@PathVariable String accountId,
                          HttpServletRequest request) {

    // Step 1: Extract and parse the token manually
    String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        throw new RuntimeException("No token");   // What status code? What for
    }
    String token = authHeader.substring(7);

    // Step 2: Validate the token — but how? Wrote this myself?
    String userId = myCustomTokenValidator.validate(token); // Hope this is cor

    // Step 3: Check permission — but this logic is duplicated in 30 other cont
    if (!userService.hasPermission(userId, "accounts:read")) {
        throw new RuntimeException("Forbidden");  // Again — what format? What
    }

    // Step 4: Finally, business logic
    return accountService.findById(accountId);
}
```

© 2026 by Innovation In Software Corporation

The Naive Approach

- Missing protections you did not think to add
  - Session fixation attacks
  - CSRF (Cross-Site Request Forgery)
  - Clickjacking via missing HTTP security headers
  - Insecure direct object references
  - These require expertise to implement correctly and are easy to get wrong
  - The fundamental flaw
    - Security is opt-in developers must remember to apply it
    - One missed endpoint is a vulnerability
    - One incorrect implementation is a vulnerability

```java
// A controller method with inline, ad-hoc security logic
@GetMapping("/api/accounts/{accountId}")
public Account getAccount(@PathVariable String accountId,
                          HttpServletRequest request) {

    // Step 1: Extract and parse the token manually
    String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        throw new RuntimeException("No token");   // What status code? What for
    }
    String token = authHeader.substring(7);

    // Step 2: Validate the token — but how? Wrote this myself?
    String userId = myCustomTokenValidator.validate(token); // Hope this is cor

    // Step 3: Check permission — but this logic is duplicated in 30 other cont
    if (!userService.hasPermission(userId, "accounts:read")) {
        throw new RuntimeException("Forbidden");  // Again — what format? What
    }

    // Step 4: Finally, business logic
    return accountService.findById(accountId);
}
```

© 2026 by Innovation In Software Corporation

Cross-Cutting Concerns

- The fundamental design failure
  - Security is a cross-cutting concern
    - Cross-cutting concerns do not belong in individual classes
    - They belong in infrastructure
  - Fundamental security principle
    - If security logic can be omitted by forgetting to call a method, it will eventually be omitted
- The correct mental model
  - Security should be something the system enforces
  - Not something developers remember to implement

Concern	Naive Approach	Framework Approach
Logging	`log.info()` in every method	AOP / logging interceptor
Transactions	`connection.commit()` in every method	`@Transactional`
Security	Auth checks in every controller	**Filter chain / AOP**

© 2026 by Innovation In Software Corporation

Spring Security

- Security as a layer, not a feature
  - Security is applied uniformly to the entire application not endpoint by endpoint
  - Developers declare rules and the framework enforces them consistently
  - A missed annotation does not create a gap because the default is to deny

Protection	Without Spring Security	With Spring Security
Authentication enforcement	Manual, per endpoint	Declared once, enforced everywhere
Authorization rules	Scattered if/else checks	Centralized rule declarations
CSRF protection	Must implement manually	On by default
Session fixation	Must implement manually	On by default
Security response headers	Must implement manually	On by default
Secure password storage	Must implement manually	Built-in `PasswordEncoder`

© 2026 by Innovation In Software Corporation

Spring Security

- The design philosophy of Spring Security
  - Principle 1: Secure by default
    - An unconfigured Spring Security application denies all access
    - You explicitly open up what should be accessible, never the reverse
    - The failure mode is "access denied," not "access granted"
  - Principle 2: Defence in depth
    - Multiple independent security layers, if one fails, another catches it
    - HTTP request authorization + method authorization + data-level authorization
    - No single point of security failure
  - Principle 3: Separation of concerns
    - Authentication (who?) and authorization (what?) are independent subsystems
    - Each is independently configurable, replaceable, and testable
    - You can swap authentication mechanisms without changing authorization logic

© 2026 by Innovation In Software Corporation

Spring Security

- The design philosophy of Spring Security
  - Principle 4: Extensibility without modification
    - Every major component is an interface with a default implementation
    - Add custom behavior by implementing the interface, not by forking the framework
    - Security policy changes are configuration changes, not code changes
  - Principle 5: Explicit over implicit
    - Security behavior should be declared and visible
    - A developer reading the security configuration should understand what is protected and how

© 2026 by Innovation In Software Corporation

Spring Security

- Important to understand the boundaries
  - Spring Security does not:
    - Make architectural decisions for you; instead, it enforces the rules you define
    - Guarantee security if you configure it incorrectly
    - Protect against vulnerabilities in your business logic
    - Replace the need for security-conscious development practices
  - Security is a system property, not a checkbox
    - Spring Security is a powerful tool only when it is configured correctly
    - Understanding how it works is what makes it effective
    - A misconfigured security framework can be worse than no framework because it creates false sense of confidence

© 2026 by Innovation In Software Corporation

Deny by Default

- Dangerous opt-in security
  - A project starts with 5 endpoints
  - All carefully annotated with security requirements
  - Six months later, a junior developer adds a new endpoint for a quick internal feature
  - In an opt-in model, this endpoint is now publicly accessible unless they remember to add the security annotation

```
All endpoints accessible
      |
      v
Developer adds @Secured to some endpoints
      |
      v
Endpoints without @Secured remain open
      |
      v
New endpoint added -> developer forgets @Secured
      |
      v
Unprotected endpoint in production  <-- VULNERABILITY
```

© 2026 by Innovation In Software Corporation

Deny by Default

- Spring Security opt-out model
  - .anyRequest().authenticated()
  - Most important line in a security configuration
    - Catch-all that ensures nothing falls through
    - Without it, any request that does not match an earlier rule may be permitted by default

```
All endpoints accessible
        |
        v
Developer adds @Secured to some endpoints
        |
        v
Endpoints without @Secured remain open
        |
        v
New endpoint added -> developer forgets @Secured
        |
        v
Unprotected endpoint in production  <-- VULNERABILITY
```

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/public/**").permitAll()   // Explicitly opened
    .requestMatchers("/actuator/health").permitAll() // Explicitly opened
    .anyRequest().authenticated()                    // Everything else: DENIED
)
```

© 2026 by Innovation In Software Corporation

Authentication and Authorization

- Authentication
  - Establishes identity
  - "I am user-123, and here is proof: a valid JWT signed by a trusted issuer"
    - Result: a populated Authentication object in the SecurityContext
  - Mechanism
    - JWT validation, username/password check, certificate validation
  - Owned by
    - AuthenticationManager -> AuthenticationProvider

```
Authentication Source          Authorization Policy
_____________________          ____________________

JWT from OAuth2 AS     ──┐
LDAP directory         ──┤  -->  Role-based rules
SAML assertion         ──┤  -->  Scope-based rules
API key                ──┘  -->  Custom permission model

Any authentication mechanism can combine with any authorization policy
```

© 2026 by Innovation In Software Corporation

Authentication and Authorization

- Authorization
  - Enforces permissions
  - "User-123 is allowed to perform GET /api/accounts/456 because they have SCOPE_accounts:read"
    - Result: request proceeds, or AccessDeniedException is thrown
  - Mechanism
    - URL pattern matching, @PreAuthorize, ACL (Access Control List)
  - Owned by
    - AuthorizationManager

```
Authentication Source          Authorization Policy
_____________________          ____________________

JWT from OAuth2 AS     ──┐
LDAP directory         ──┤  -->  Role-based rules
SAML assertion         ──┤  -->  Scope-based rules
API key                ──┘  -->  Custom permission model

Any authentication mechanism can combine with any authorization policy
```

© 2026 by Innovation In Software Corporation

Architecture Overview

© 2026 by Innovation In Software Corporation

The Two Enforcement Layers

- Key points
  - The two layers are independent
    - Either can be active without the other
  - The two layers are complementary
    - Using both provides defence in depth
  - They share the same SecurityContext
    - Authentication happens once and both layers use the result

```
HTTP Request
     |
     ▼
┌──────────────────────────────────┐
│        LAYER 1: WEB LAYER        │
│        Security Filter Chain     │
│                                  │
│  Operates on: HTTP requests      │
│  Asks: is this request allowed   │
│        to enter the application?│
│                                  │
│  Configured via:                 │
│  SecurityFilterChain bean        │
└──────────────────────────────────┘
                  |
                  | Allowed
                  ▼
     Your Controllers & Services
                  |
                  ▼
┌──────────────────────────────────┐
│     LAYER 2: METHOD LAYER        │
│     AOP Method Security          │
│                                  │
│  Operates on: Java method calls  │
│  Asks: is this identity allowed  │
│        to call this method?      │
│                                  │
│  Configured via:                 │
│  @PreAuthorize annotations       │
└──────────────────────────────────┘
```

© 2026 by Innovation In Software Corporation

The Authentication Object

- The Authentication object is the most important data structure in Spring Security
  - Every security decision in the filter chain and in method security —is made by reading this object
- The fields
  - GetPrincipal()
    - Before authentication, this is the raw credential (the token string)
    - After authentication, this is the verified identity (the sub claim from the JWT, or a UserDetails object for form login)
  - GetCredentials()
    - The proof of identity
    - For JWT auth this is the token.
    - Spring Security clears this after authentication for security because(you don't want the token sitting in memory longer than necessary)

```
public interface Authentication extends Principal, Serializable {

    Object getPrincipal();           // WHO — the identity (username, user obje
    Object getCredentials();         // PROOF — the token, password (cleared af
    Collection<? extends
        GrantedAuthority> getAuthorities(); // WHAT — roles and permissions
    boolean isAuthenticated();       // STATE — has this been verified?
    Object getDetails();             // CONTEXT — IP address, session ID, etc.
}
```

```
Before authentication:                After authentication:

┌──────────────────────┐              ┌──────────────────────┐
│ principal: token     │              │ principal: "alice"   │
│ credentials: token   │ ──────►      │ credentials: null    │
│ authorities: []      │  Verified    │ authorities:         │
│ authenticated: false │              │   [SCOPE_read,       │
└──────────────────────┘              │    ROLE_USER]        │
                                      │ authenticated: true  │
                                      └──────────────────────┘
```

© 2026 by Innovation In Software Corporation

The Authentication Object

- The fields
  - GetAuthorities()
    - The set of permissions granted to this identity.
    - This is what hasRole() and hasAuthority() check against
  - IsAuthenticated()
    - The flag that distinguishes "we have a token" from "we have verified the token"
- The authentication process
  - Takes an unverified Authentication object
  - Returns a verified one with the principal resolved and authorities populated

```
public interface Authentication extends Principal, Serializable {

    Object getPrincipal();           // WHO — the identity (username, user obje
    Object getCredentials();         // PROOF — the token, password (cleared af
    Collection<? extends
        GrantedAuthority> getAuthorities(); // WHAT — roles and permissions
    boolean isAuthenticated();       // STATE — has this been verified?
    Object getDetails();             // CONTEXT — IP address, session ID, etc.
}
```

```
Before authentication:                After authentication:

┌──────────────────────┐              ┌──────────────────────┐
│ principal: token     │              │ principal: "alice"   │
│ credentials: token   │ ──────►      │ credentials: null    │
│ authorities: []      │  Verified    │ authorities:         │
│ authenticated: false │              │   [SCOPE_read,       │
└──────────────────────┘              │    ROLE_USER]        │
                                      │ authenticated: true  │
                                      └──────────────────────┘
```

© 2026 by Innovation In Software Corporation

Roles vs Authorities

- Roles
  - Represent broad categories of users
    - ADMIN, USER, MANAGER
  - Spring Security automatically adds the ROLE_ prefix internally
  - hasRole('ADMIN') checks for the authority ROLE_ADMIN
  - Declared as: ROLE_ADMIN, ROLE_USER

```
JWT Token:                                  SecurityContext authorities:
{                                           [
  "sub": "alice",                             "SCOPE_read:accounts",
  "scope": "read:accounts                    "SCOPE_write:accounts",
            write:accounts",   ──►            "ROLE_USER"
  "roles": ["USER"]                         ]
}
```

© 2026 by Innovation In Software Corporation

Roles vs Authorities

- Authorities
  - Represent specific permissions
    - read:accounts, write:transactions
  - No prefix convention, checked exactly as stored
  - hasAuthority('SCOPE_read:accounts')
    - Checks for that exact string
  - OAuth2 scopes are stored with the SCOPE_ prefix automatically

```
JWT Token:                                SecurityContext authorities:
{                                         [
  "sub": "alice",                           "SCOPE_read:accounts",
  "scope": "read:accounts                  "SCOPE_write:accounts",
            write:accounts",   ──►          "ROLE_USER"
  "roles": ["USER"]                       ]
}
```

© 2026 by Innovation In Software Corporation

Roles vs Authorities

- The ROLE_ prefix behavior
  - hasRole('ADMIN')
  - hasAuthority('ROLE_ADMIN')
  - Are equivalent
  - But hasRole('ROLE_ADMIN')
    - Would check for ROLE_ROLE_ADMIN

```
JWT Token:                                SecurityContext authorities:
{                                         [
  "sub": "alice",                           "SCOPE_read:accounts",
  "scope": "read:accounts                  "SCOPE_write:accounts",
             write:accounts",   ──►         "ROLE_USER"
  "roles": ["USER"]                       ]
}
```

© 2026 by Innovation In Software Corporation

Roles vs Authorities

- OAuth2/JWT context, the convention is
  - Scopes from the token are stored as
    - SCOPE_<scope_name>
    - use hasAuthority('SCOPE_read:accounts')
  - Application roles if present in the tokenare stored as
    - ROLE_<role_name>
    - use hasRole('USER')

```
JWT Token:                                SecurityContext authorities:
{                                         [
  "sub": "alice",                           "SCOPE_read:accounts",
  "scope": "read:accounts                  "SCOPE_write:accounts",
             write:accounts",   ──►         "ROLE_USER"
  "roles": ["USER"]                       ]
}
```

© 2026 by Innovation In Software Corporation

SecurityContext

- Where the Authentication object lives during a request
- The ThreadLocal strategy
  - Each thread has its own SecurityContext
  - The filter chain populates it at the start of the request
  - It is available throughout the request lifecycle
  - It is cleared when the request completes

```
┌──────────────────────────────────────────────────────────────┐
│                      SecurityContextHolder                   │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │                  SecurityContext                     │    │
│  │                                                      │    │
│  │   Authentication {                                   │    │
│  │     principal: "alice"                               │    │
│  │     authorities: [SCOPE_read, ROLE_USER]             │    │
│  │     authenticated: true                              │    │
│  │   }                                                  │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  Storage strategy: ThreadLocal (default)                     │
│  Scope: per request — cleared at end of every request        │
└──────────────────────────────────────────────────────────────┘
```

```java
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String currentUser = auth.getName();                    // the principal name
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();
```

© 2026 by Innovation In Software Corporation

SecurityContext

- Where the Authentication object lives during a request
- The ThreadLocal strategy
  - Each thread has its own SecurityContext
  - The filter chain populates it at the start of the request
  - It is available throughout the request lifecycle
  - It is cleared when the request completes

```
┌──────────────────────────────────────────────────────────────┐
│                      SecurityContextHolder                   │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │                  SecurityContext                     │    │
│  │                                                      │    │
│  │   Authentication {                                   │    │
│  │     principal: "alice"                               │    │
│  │     authorities: [SCOPE_read, ROLE_USER]             │    │
│  │     authenticated: true                              │    │
│  │   }                                                  │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  Storage strategy: ThreadLocal (default)                     │
│  Scope: per request — cleared at end of every request        │
└──────────────────────────────────────────────────────────────┘
```

```java
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
String currentUser = auth.getName();                    // the principal name
Collection<? extends GrantedAuthority> authorities = auth.getAuthorities();
```

© 2026 by Innovation In Software Corporation

Major Components

- The indentation shows delegation relationships
  - The filter chain contains filters
    - Authentication filters delegate to the AuthenticationManager
    - The AuthenticationManager delegates to AuthenticationProvider(s)
  - Method security uses the same AuthorizationManager concept as the filter chain
    - But invoked via AOP rather than a filter
  - The shared SecurityContext connects both layers
    - Populated by Layer 1 and read by Layer 2

```
┌──────────────────────────────────────────────────────────────────────┐
│                        LAYER 1: FILTER CHAIN                         │
│                                                                      │
│  SecurityFilterChain                                                 │
│  └── Filter 1: SecurityContextPersistenceFilter                      │
│  └── Filter 2: Authentication Filter (JWT / OAuth2 / Basic...)       │
│         └── AuthenticationManager                                    │
│               └── AuthenticationProvider(s)                          │
│                     └── populates SecurityContext                    │
│  └── Filter 3: ExceptionTranslationFilter                            │
│         └── AuthenticationEntryPoint  (handles 401)                  │
│         └── AccessDeniedHandler       (handles 403)                  │
│  └── Filter 4: AuthorizationFilter                                   │
│         └── AuthorizationManager                                     │
│               └── reads SecurityContext                              │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│                      LAYER 2: METHOD SECURITY                        │
│                                                                      │
│  @EnableMethodSecurity                                               │
│  └── AOP Proxy (wraps your @Service beans)                           │
│        └── MethodSecurityInterceptor                                 │
│              └── AuthorizationManager                                │
│                    └── evaluates @PreAuthorize SpEL expression       │
│                    └── reads SecurityContext (same Authentication)   │
└──────────────────────────────────────────────────────────────────────┘
```

© 2026 by Innovation In Software Corporation

Component Responsibilities

- The pattern
  - extract → verify → store → enforce
  - Extract
    - The authentication filter extracts credentials from the request
  - Verify
    - The AuthenticationProvider verifies the credentials
  - Store
    - The verified Authentication is stored in the SecurityContext
  - Enforce
    - The AuthorizationManager (in the filter chain or via AOP) enforces access rules

Component	Layer	Responsibility
`SecurityFilterChain`	Web	Container for the ordered filter sequence
Authentication Filter	Web	Extracts credentials from the HTTP request
`AuthenticationManager`	Web	Coordinates the authentication process
`AuthenticationProvider`	Web	Performs actual credential verification
`SecurityContext`	Both	Stores the verified `Authentication` for the request
`SecurityContextHolder`	Both	Provides access to the `SecurityContext`
`AuthorizationManager`	Both	Evaluates access rules against the `Authentication`
`AuthenticationEntryPoint`	Web	Produces the 401 response on auth failure
`AccessDeniedHandler`	Web	Produces the 403 response on authorization failure
`ExceptionTranslationFilter`	Web	Routes security exceptions to the right handler
AOP Proxy	Method	Intercepts method calls for security checks
`@PreAuthorize`	Method	Declares the authorization rule for a method

© 2026 by Innovation In Software Corporation

Complete Request Lifecycle

- Steps 1-2
  - Extract and Verify
    - Authentication Filter + AuthenticationProvider
- Step 3:
  - Store SecurityContext
- Step 4:
  - Enforce at web layer
    - AuthorizationManager in filter chain

```
HTTP Request: GET /accounts/123
Authorization: Bearer <jwt>
        |
        ▼
┌──────────────────────────────────────────────────────────┐
│                  Security Filter Chain                   │
│                                                          │
│  1. Extract Bearer token from Authorization header       │
│            |                                             │
│  2. Verify JWT signature, expiry, issuer                 │
│            |                                             │
│  3. Populate SecurityContext:                            │
│     Authentication {                                     │
│       principal: "alice", authenticated: true            │
│       authorities: [SCOPE_read:accounts, ROLE_USER]      │
│     }                                                    │
│            |                                             │
│  4. Check URL rule: /accounts/** → authenticated()       │
│     ✅ alice is authenticated — proceed                  │
└──────────────────────────────────────────────────────────┘
                        |
                        ▼
                  AccountController
                  getAccount("123")
```

© 2026 by Innovation In Software Corporation

Complete Request Lifecycle

- Steps 5-6
  - Enforce at method layer
    - AOP Proxy + AuthorizationManager
- SecurityContext
  - Is populated once (step 3) and read by both enforcement points (steps 4 and 6)
  - This is the shared state that connects both layers.

```
                    ▼
            AccountController
            getAccount("123")
                    |
                    ▼
┌──────────────────────────────────────────────────────┐
|                  AOP Proxy                           |
|                                                      |
|  5. Intercept call to AccountService.getAccount()    |
|            |                                         |
|  6. Evaluate @PreAuthorize:                          |
|     "hasAuthority('SCOPE_read:accounts')"            |
|     Read SecurityContext → alice has SCOPE_read:accounts|
|     ✅ Authorized — proceed                          |
└──────────────────────────────────────────────────────┘
                    |
                    ▼
          AccountService.getAccount("123")
          — your code runs here —
```

© 2026 by Innovation In Software Corporation

The Filter Chain

© 2026 by Innovation In Software Corporation

The Filter Chain

- A chain of servlet filters registered with the web container
  - Spring Security registers a FilterChainProxy with the servlet container
  - FilterChainProxy holds one or more SecurityFilterChain instances
  - Each SecurityFilterChain is an ordered list of security filters
  - Every HTTP request passes through the chain before reaching your code

```
Request -> [Filter 1] -> [Filter 2] -> [Filter N] -> Servlet
(Controller)
               |              |              |
            Can reject    Can enrich    Can authorize
            (401)         (add context)  (403)
```

© 2026 by Innovation In Software Corporation

The Servlet Container

- The Servlet specification defines the contract
  - Every HTTP request is handled by the container on a dedicated thread
  - Before the request reaches any servlet, the container runs it through a Filter Chain
  - Filters are registered with the container and form an ordered pipeline
  - This is standard Jakarta EE, not Spring-specific
- Why this matters for Spring Security
  - Spring Security is implemented entirely as Servlet Filters
  - It operates inside an established, well-understood specification
  - Security runs before Spring MVC, before controllers, before your code

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- Spring Security
  - Uses the Servlet filter as its foundational mechanism
- Any filter can short-circuit the chain
  - Authentication failure
    - Filter returns 401, chain stops, controller never invoked
  - Authorization failure
    - Filter returns 403, chain stops, controller never invoked
  - Success
    - Chain completes, request reaches controller with security context established

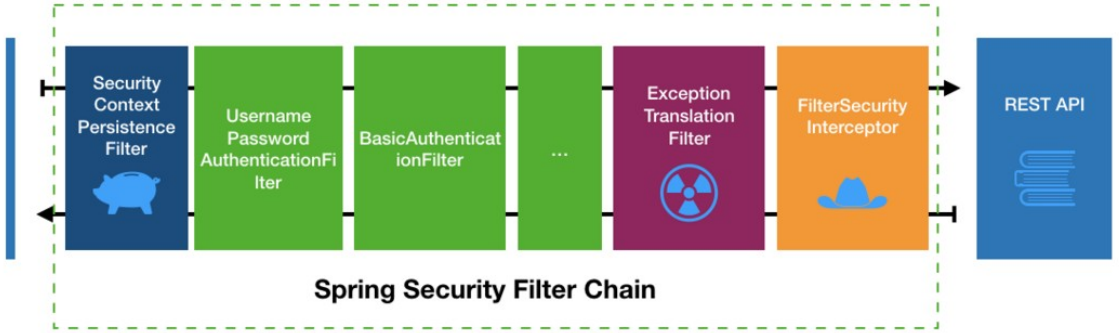


Image Credit: https://segmentfault.com/a/1190000021690783

© 2026 by Innovation In Software Corporation

Default Filter Chain

- Spring Boot configures a default SecurityFilterChain automatically
- Adding spring-boot-starter-security immediately activates
  - The default configuration:
  - All endpoints require authentication
  - Form login is enabled
  - HTTP Basic authentication is enabled
  - CSRF protection is enabled
  - Session management is enabled

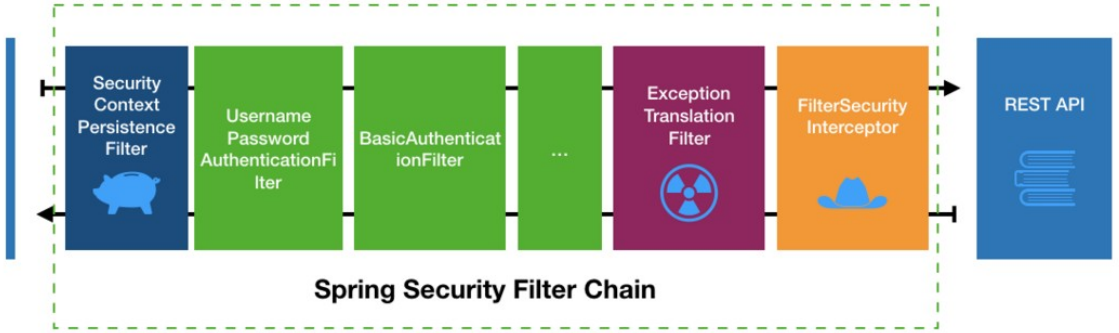


Image Credit: https://segmentfault.com/a/1190000021690783

© 2026 by Innovation In Software Corporation

Default Filter Chain

- Starting an application with no security configuration
  - Spring Boot generates a random password for the default user user
  - All requests redirect to a generated login form
  - This is the "secure by default" behavior
- There is no such thing as "Spring Security is not configured yet"
  - The moment the dependency is on the classpath, everything is locked down
  - Your job is to replace the default configuration, not add to it

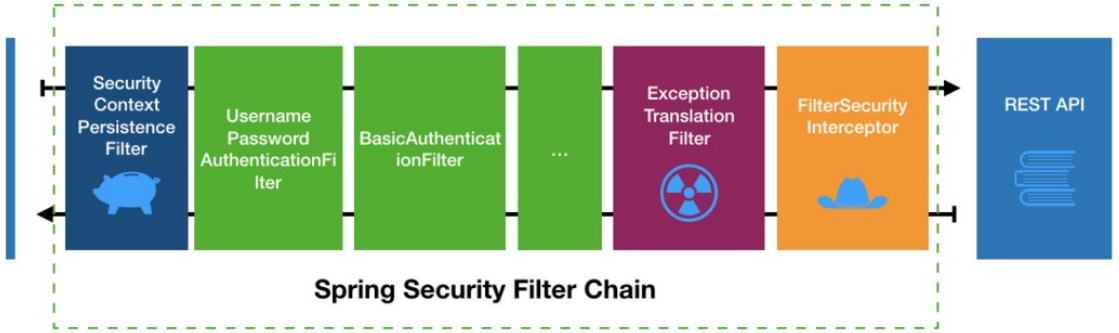


Image Credit: https://segmentfault.com/a/1190000021690783

© 2026 by Innovation In Software Corporation

Default Filter Chain

- The practical implication
  - When you declare your own SecurityFilterChain bean
  - Spring Boot's auto-configured default is backed off entirely
- Your bean completely replaces it
  - This means your configuration must handle everything the default was handling
  - Including things like the security response headers that are added automatically.

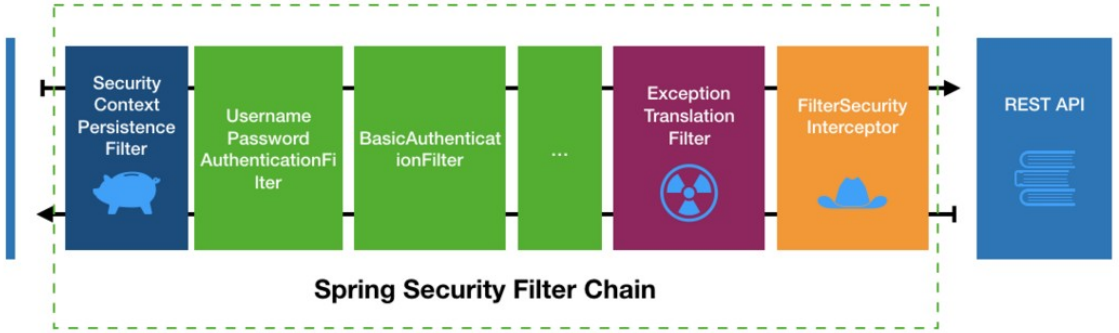


Image Credit: https://segmentfault.com/a/1190000021690783

© 2026 by Innovation In Software Corporation

Filter Chain

- HttpSecurity is a DSL
  - A fluent builder that registers filters
  - Each method call registers one or more filters like
    - authorizeHttpRequests, oauth2ResourceServer
  - The filters are added to the chain in a fixed, predetermined order
  - http.build() assembles the final SecurityFilterChain

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- Step by step
  - authorizeHttpRequests
    - Registers the AuthorizationFilter
    - The rules are evaluated in order, first match wins
    - Always put specific rules before general ones.
  - oauth2ResourceServer with jwt
    - Registers the BearerTokenAuthenticationFilte
    - Configures the JWT AuthenticationProvider
    - This is the line that activates JWT authentication

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- Step by step
  - sessionCreationPolicy(STATELESS)
    - Tells Spring Security not to create or use HTTP sessions
    - For a REST API with Bearer tokens, sessions are not needed and create unnecessary state
  - csrf.disable()
    - Disables CSRF protection
  - Each method maps to one or more filters being added to the chain

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- HTTP sessions and REST APIs
  - Traditional web apps
    - User logs in once, session stores identity, requests use session cookie
  - REST APIs with Bearer tokens
    - Every request carries its own credential
  - Sessions are unnecessary and create problems
    - Server-side state conflicts with horizontal scaling
    - Session cookies are vulnerable to CSRF attacks
    - Sessions must be managed, expired, and invalidated

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- What STATELESS does
  - Spring Security will never create an HTTP session
  - Spring Security will never use an existing HTTP session
  - Each request must be authenticated independently via its Bearer token
- No server-side session state is maintained
  - Eliminates an entire class of session-based attacks
  - Forces each request to prove its own identity
  - Makes the application horizontally scalable because no sticky sessions needed

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- Why Spring Security enables CSRF protection by default
  - Protects traditional web applications that use session cookies
  - The CSRF token must be included in every state-changing request
  - Without it, the browser's automatic cookie inclusion creates a vulnerability
- When it is safe to disable CSRF
  - Your API uses Bearer tokens (not cookies) for authentication
  - Bearer tokens are not sent automatically by the browser BUT must be explicitly included
    - A malicious page cannot forge a request because it cannot access the Bearer token

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            // Authorization rules
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated()
            )

            // Authentication mechanism
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())
            )

            // REST API configuration
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .csrf(csrf -> csrf.disable());

        return http.build();
    }
}
```

© 2026 by Innovation In Software Corporation

Filter Chain

- JWT authentication in Spring Security
  - Steps 7-11 are what the JwtAuthenticationProvider does internally
  - The JWKS (JSON Web Key Set) is fetched from the authorization server's JWKS endpoint
    - configured via spring.security.oauth2.resourceserver.jwt.jwks-uri
  - The keys are cached, not fetched on every request
  - All validation happens before the Authentication is considered valid
    - Validation includes signature, expiry, issuer

```
Request: Authorization: Bearer <token>
        |
        ▼
┌──────────────────────────────────────┐
│  BearerTokenAuthenticationFilter     │
│                                      │
│  1. Read Authorization header        │
│  2. Extract token string             │
│  3. Create unauthenticated           │
│     BearerTokenAuthenticationToken   │
│  4. Pass to AuthenticationManager    │
└──────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────┐
│  ProviderManager (AuthenticationManager│
│                                      │
│  5. Find a provider that supports    │
│     BearerTokenAuthenticationToken   │
│ 6. Delegate to JwtAuthenticationProvider│
└──────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────┐
│  JwtAuthenticationProvider           │
│                                      │
│  7.  Fetch JWKS from auth server     │
│  8.  Verify token signature          │
│  9.  Check expiry (exp claim)        │
│  10. Check issuer (iss claim)        │
│  11. Extract authorities from claims │
│  12. Return verified Authentication  │
└──────────────────────────────────────┘
                  |
                  ▼
┌──────────────────────────────────────┐
│  SecurityContext                     │
│  13. Store verified Authentication   │
│      → chain continues               │
└──────────────────────────────────────┘
```

© 2026 by Innovation In Software Corporation

Filter Chain

- JWT authentication in Spring Security
  - If any step fails, the provider throws an AuthenticationException
    - Caught by the ExceptionTranslationFilter
    - Converted into a 401 response
    - Triggered by faults like invalid signature, expired token, wrong issuer

```
Request: Authorization: Bearer <token>
        |
        ▼
┌──────────────────────────────────────┐
│   BearerTokenAuthenticationFilter    │
│                                      │
│   1. Read Authorization header       │
│   2. Extract token string            │
│   3. Create unauthenticated          │
│      BearerTokenAuthenticationToken  │
│   4. Pass to AuthenticationManager   │
└──────────────────────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────┐
│   ProviderManager (AuthenticationManager│
│                                      │
│   5. Find a provider that supports   │
│      BearerTokenAuthenticationToken  │
│ 6. Delegate to JwtAuthenticationProvider│
└──────────────────────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────┐
│   JwtAuthenticationProvider          │
│                                      │
│   7.  Fetch JWKS from auth server    │
│   8.  Verify token signature         │
│   9.  Check expiry (exp claim)       │
│   10. Check issuer (iss claim)       │
│   11. Extract authorities from claims│
│   12. Return verified Authentication │
└──────────────────────────────────────┘
                   |
                   ▼
┌──────────────────────────────────────┐
│   SecurityContext                    │
│   13. Store verified Authentication  │
│       → chain continues              │
└──────────────────────────────────────┘
```

© 2026 by Innovation In Software Corporation

Filter Chain

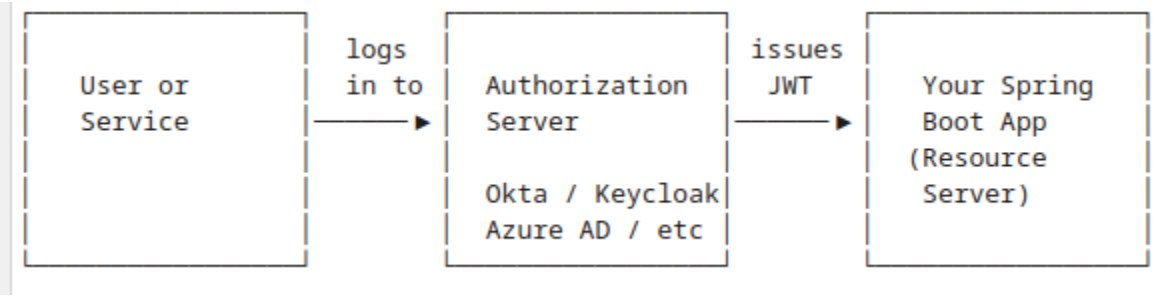
- Configuring JWT authentication
  - Minimum configuration to accept JWT Bearer tokens
  - What this configuration does
    - jwks-uri: where to fetch signing keys to verify token signatures
    - issuer-uri: the expected value of the iss claim in tokens; tokens from other issuers are rejected

```yaml
# application.yml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          jwks-uri: https://auth.example.com/.well-known/jwks.json
          issuer-uri: https://auth.example.com
```

© 2026 by Innovation In Software Corporation

Authorization Server

- The Authorization Server
  - Responsible for one thing: authenticating identities and issuing tokens
  - Your Spring Boot application never sees usernames or passwords
  - It only ever sees the JWT token that the Authorization Server already verified and signed



© 2026 by Innovation In Software Corporation

Authorization Server

- The jwks-uri
  - Is a public URL hosted by the Authorization Server
    - Publishes its public keys in a standard format called JWKS (JSON Web Key Set
  - When your Spring Boot app receives a JWT,
    - Fetches the public keys from the jwks-uri
    - Uses those keys to verify the token's signature
    - If the signature is valid, the token genuinely came from that Authorization Server and has not been tampered with

```
Authorization Server                    Your Spring Boot App
hosts:                                  on startup / first request:

GET https://auth.example.com            Spring Security fetches
    /.well-known/jwks.json      ◄──────  the public keys and
                                        caches them


{                                       On every request:
  "keys": [{                            Spring Security uses the
    "kty": "RSA",                       cached keys to verify
    "kid": "key-1",                     the JWT signature
    "n": "...",  ← public key
    "e": "AQAB"
  }]
}
```

© 2026 by Innovation In Software Corporation

Authorization Server

- The issuer-uri
  - A JWT contains a claim called iss (issuer)
    - String that identifies which Authorization Server created the token.
  - The issuer-uri you configure tells Spring Security
    - “I will only accept tokens that were issued by this specific Authorization Server”
  - In a large organization there may be many Authorization Servers
    - Without this check, a token issued for one system could potentially be used to access another

```
JWT payload:
{
  "iss": "https://auth.example.com",   ← who issued this token
  "sub": "alice",
  "exp": 1234567890,
  "scope": "read:accounts"
}

Your config:
  issuer-uri: https://auth.example.com  ← only accept tokens from here

Spring Security checks: does the token's "iss" match my configured issuer-uri?
Match → proceed
 No match → reject with 401
```

© 2026 by Innovation In Software Corporation

Authorization Server

- In practice, almost nobody builds their own AS
- In a large bank for example
  - very common to have an enterprise-managed Keycloak or similar platform
  - The security team runs it
  - A developer asks them for the jwks-uri and issuer-uri values for their configuration

Option	Examples	Typical use case
Cloud identity provider	Okta, Auth0, Azure AD, AWS Cognito	SaaS, cloud-native enterprise
Self-hosted open source	Keycloak, Spring Authorization Server	On-premise, regulated environments
Enterprise identity platform	Ping Identity, ForgeRock	Large enterprise, legacy integration

© 2026 by Innovation In Software Corporation

The Complete AS Picture

```
Security Team / Platform Team
runs the Authorization Server (e.g. Keycloak)
and gives you two URLs:

  jwks-uri:   https://auth.internal.example.com/realms/myapp/protocol/openid-connect/certs
  issuer-uri: https://auth.internal.example.com/realms/myapp

You put them in application.yml:

  spring:
    security:
      oauth2:
        resourceserver:
          jwt:
            jwks-uri:   https://auth.internal.example.com/...
            issuer-uri: https://auth.internal.example.com/realms/myapp

Spring Security does the rest — fetches keys, verifies signatures,
checks issuers, rejects invalid tokens.
Your application code never handles a username or password.
```

© 2026 by Innovation In Software Corporation

Claims Mapping

- Authorization servers
  - Are not standardized in how they represent roles and scopes
    - Some use scope (space-separated), some use roles (array), some use permissions
  - The converter is where you normalize this into Spring Security's authority format
  - A common enterprise pattern
    - Both scopes and roles in the token, mapped to different authority prefixes (SCOPE_ for scopes, ROLE_ for roles)
    - Then combine them into a single authority list
    - This requires a custom JwtGrantedAuthoritiesConverter

```java
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter authoritiesConverter =
        new JwtGrantedAuthoritiesConverter();

    authoritiesConverter.setAuthoritiesClaimName("roles");     // use "roles" claim
    authoritiesConverter.setAuthorityPrefix("ROLE_");          // prefix with ROLE_

    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authoritiesConverter);
    return converter;
}
```

© 2026 by Innovation In Software Corporation

Authorization

- The AuthorizationFilter evaluates URL-based access rules
  - Runs after authentication
  - The SecurityContext is already populated
    - Reads the Authentication object from the SecurityContext
  - Evaluates rules declared in authorizeHttpRequests()
  - Rules are evaluated in order
    - First match wins

```
.authorizeHttpRequests(auth -> auth
    // Most specific rules first
    .requestMatchers("/actuator/health").permitAll()
    .requestMatchers("/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/accounts/**")
        .hasAuthority("SCOPE_read:accounts")
    .requestMatchers(HttpMethod.POST, "/accounts/**")
        .hasAuthority("SCOPE_write:accounts")
    .requestMatchers("/admin/**").hasRole("ADMIN")

    // Catch-all last — any unmatched request requires authentication
    .anyRequest().authenticated()
)
```

© 2026 by Innovation In Software Corporation

Authorization

- First match wins
  - Common source of bugs, for example
    - .anyRequest().authenticated()
    - before .requestMatchers("/public/**").permitAll(),
    - The anyRequest rule matches first and the public rule is never evaluated
- Best practice for rule ordering
  - Specific path + method combinations first
  - Broad path patterns next
  - .anyRequest() always last as a catch-all

```
.authorizeHttpRequests(auth -> auth
    // Most specific rules first
    .requestMatchers("/actuator/health").permitAll()
    .requestMatchers("/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/accounts/**")
        .hasAuthority("SCOPE_read:accounts")
    .requestMatchers(HttpMethod.POST, "/accounts/**")
        .hasAuthority("SCOPE_write:accounts")
    .requestMatchers("/admin/**").hasRole("ADMIN")

    // Catch-all last — any unmatched request requires authentication
    .anyRequest().authenticated()
)
```

© 2026 by Innovation In Software Corporation

Authorization

- Distinction
  - The distinction between HTTP method in the matcher and just the path is important for REST APIs
    - A GET /accounts (read) might require SCOPE_read:accounts
    - While a POST /accounts (write) requires SCOPE_write:accounts
    - Combining method and path in the matcher enables this

Expression	Meaning
`permitAll()`	Anyone, including anonymous
`authenticated()`	Any authenticated identity
`hasRole("ADMIN")`	Has role `ROLE_ADMIN`
`hasAuthority("SCOPE_read")`	Has this exact authority
`hasAnyRole("ADMIN","USER")`	Has any of these roles
`denyAll()`	Nobody — useful for disabling endpoints

© 2026 by Innovation In Software Corporation

Authorization

- Exception handling: 401 vs 403
  - Two distinct failure modes require two distinct handlers
- ExceptionTranslationFilter
  - The traffic controller
  - After authentication but before the application
  - Catches
    - AuthenticationException → delegates to AuthenticationEntryPoint
    - Catches AccessDeniedException → delegates to AccessDeniedHandler
    - Ensures the right HTTP status code is always returned

```
  Request arrives
        |
        ▼
  Authentication fails        ──▶  AuthenticationEntryPoint  ──▶  401
Unauthorized
  (no token, invalid token)        "You need to authenticate"

  Authentication succeeds
  Authorization fails         ──▶  AccessDeniedHandler       ──▶  403
Forbidden
  (wrong role/scope)               "You are not allowed"
```

© 2026 by Innovation In Software Corporation

Authorization

- AuthenticationEntryPoint
  - The 401 responder interface with one job
    - Write the HTTP response when a request arrives with no valid identity
  - This is the entry point into the authentication process
    - "You need to prove who you are before going further"
  - Does not perform authentication
    - Tells the caller what they need to do
    - In a browser-based app, the default implementation redirects to a login page

```
  Request arrives
        |
        ▼
  Authentication fails        ──▶  AuthenticationEntryPoint  ──▶  401
Unauthorized
  (no token, invalid token)        "You need to authenticate"

  Authentication succeeds
  Authorization fails         ──▶  AccessDeniedHandler       ──▶  403
Forbidden
  (wrong role/scope)               "You are not allowed"
```

© 2026 by Innovation In Software Corporation

Authorization

- AuthenticationEntryPoint
- When it fires
  - No Authorization header at all
  - Token is present but malformed
    - It cannot be parsed
  - Token signature is invalid
    - It was not signed by the expected Authorization Server
  - Token is expired

```
  Request arrives
        |
        ▼
  Authentication fails        ──►  AuthenticationEntryPoint  ──►  401
Unauthorized
  (no token, invalid token)        "You need to authenticate"

  Authentication succeeds
  Authorization fails         ──►  AccessDeniedHandler       ──►  403
Forbidden
  (wrong role/scope)               "You are not allowed"
```

© 2026 by Innovation In Software Corporation

Authorization

- AccessDeniedHandler
  - The 403 responder interface with one job:
    - Write the HTTP response when an authenticated identity tries to do something it is not permitted to do
  - The caller has already proved who they are
  - The problem is that who they are is not enough
  - The token is valid and trusted
    - It simply does not carry the required role or scope for this operation
    - Retrying with the same token will never succeed because the identity itself lacks permission

```
  Request arrives
       |
       ▼
  Authentication fails        ──▶  AuthenticationEntryPoint  ──▶  401
Unauthorized
  (no token, invalid token)        "You need to authenticate"

  Authentication succeeds
  Authorization fails         ──▶  AccessDeniedHandler       ──▶  403
Forbidden
  (wrong role/scope)               "You are not allowed"
```

© 2026 by Innovation In Software Corporation

Authorization

- AccessDeniedHandler
- When it fires
  - Token is valid but is missing a required scope
    - e.g. SCOPE_write:accounts is needed but only SCOPE_read:accounts is present
    - Token is valid but the user lacks the required role
    - e.g. ROLE_ADMIN is required but the user has ROLE_USER
  - @PreAuthorize on a service method evaluates to false
    - More on this later

```
401 → "I don't know who you are"
      Fix: get a valid token and try again

403 → "I know who you are, but you can't do this"
      Fix: get a token with different scopes, or contact an admin
           Retrying will not help
```

© 2026 by Innovation In Software Corporation

Authorization

- These two filters can both be overridden with customized implementations

```
.exceptionHandling(ex -> ex
    .authenticationEntryPoint((request, response, authException) -> {
        response.setStatus(HttpStatus.UNAUTHORIZED.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write("""
            {"error": "authentication_required",
             "message": "A valid Bearer token is required"}
        """);
    })
    .accessDeniedHandler((request, response, accessDeniedException) -> {
        response.setStatus(HttpStatus.FORBIDDEN.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write("""
            {"error": "access_denied",
             "message": "Insufficient permissions"}
        """);
    })
)
```

© 2026 by Innovation In Software Corporation

Security Headers

- Spring Security adds protective HTTP headers to every response automatically
  - You do not need to configure these
  - They apply to every response automatically
  - When you declare a custom SecurityFilterChain, these are still included
  - You can add custom headers

Header	What it does	Default value
`X-Content-Type-Options`	Prevents MIME sniffing attacks	`nosniff`
`X-Frame-Options`	Prevents clickjacking via iframes	`DENY`
`X-XSS-Protection`	Legacy XSS filter hint for older browsers	`0`
`Cache-Control`	Prevents caching of secure responses	`no-cache, no-store`
`Strict-Transport-Security`	Enforces HTTPS (HSTS)	`max-age=31536000`

© 2026 by Innovation In Software Corporation

The Filter Chain

- The filter chain
  - Every filter in the chain has one job
    - The chain works because each filter trusts that the filters before it have already done their job
  - By the time a request reaches your controller, Spring Security has already
    - Established session context
    - Extracted and verified the token
    - Populated the SecurityContext
    - Checked URL-level authorization
  - Customizing the chain means inserting your own step into that pipeline
    - At a position where the things you need are already done, and the things that depend on you have not run yet

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- The three practical reasons to add a custom filter
- You need something done before authentication
  - Validating a required request header before wasting time on token verification
  - Rejecting requests from blocked IP addresses
  - Enforcing API versioning rules

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- The three practical reasons to add a custom filter
- You need something done after authentication but before authorization
  - Resolving a tenant in a multi-tenant system using the verified identity
  - Enriching the SecurityContext with additional data fetched from a database
  - Rate limiting per authenticated user

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- The three practical reasons to add a custom filter
- You need something done after authorization
  - Audit logging; for example, log requests that passed all security checks
  - Adding response headers based on the authenticated identity

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- The practical reality
  - In most applications you will rarely add a custom filter
- The cases where you do are almost always one of
  - A cross-cutting technical concern that has no better home
    - Audit logging, rate limiting, tenant resolution
  - An integration requirement imposed by your organization
    - A proprietary header that must be validated on every request

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- Key positions in the filter chain
  - Before BearerTokenAuthenticationFilter
    - Pre-authentication validation
  - After BearerTokenAuthenticationFilter
    - Post-authentication, pre-authorization
  - After AuthorizationFilter
    - Ppost-authorization, before application

```java
public class RequestValidationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        // Custom logic — e.g., validate a required header
        String apiVersion = request.getHeader("X-Api-Version");
        if (apiVersion == null) {
            response.sendError(HttpStatus.BAD_REQUEST.value(),
                "X-Api-Version header is required");
            return;  // halt the chain
        }

        filterChain.doFilter(request, response);  // continue the chain
    }
}
```

```java
 .addFilterBefore(new RequestValidationFilter(),
     BearerTokenAuthenticationFilter.class)  // run before authentication

 .addFilterAfter(new AuditLoggingFilter(),
     AuthorizationFilter.class)              // run after authorization
```

© 2026 by Innovation In Software Corporation

The Filter Chain

- What the filter chain gives us
  - Authentication: we know who is making the request
  - URL-level authorization: broad rules about which endpoints are accessible
  - The populated SecurityContext available throughout the request
- What the filter chain cannot do
  - Enforce authorization rules based on method arguments
  - Enforce authorization on service methods called from non-HTTP entry points
    - Kafka consumers, scheduled jobs, internal orchestration
  - Express fine-grained ownership rules
    - "This user can only access their own resources"

```
// The filter chain can say: GET /accounts/** requires SCOPE_read:accounts
// The filter chain cannot say: you can only get YOUR OWN account

// Method security can say both:
@PreAuthorize("""
    hasAuthority('SCOPE_read:accounts')
    && #accountOwnerId == authentication.name
    """)
public Account getAccount(String accountOwnerId, String accountId) { ... }
```

© 2026 by Innovation In Software Corporation

Method Security

© 2026 by Innovation In Software Corporation

Method Security

- AOP Proxies
  - Spring wraps your service beans in a proxy at startup
  - Every call goes through the proxy transparently
  - The proxy is invisible, callers do not know it is there

```
Application Context at startup:

  You declare:                        Spring creates:

  ┌──────────────────┐                ┌──────────────────────────┐
  │ AccountService   │ ──────►        │ AOP Proxy                │
  │ @Service         │                │ (wraps AccountService)   │
  │                  │                │                          │
  │ getAccount()     │                │ getAccount() {           │
  │ createAccount()  │                │   check @PreAuthorize    │
  │ deleteAccount()  │                │   if ok → AccountService │
  └──────────────────┘                │             .getAccount() │
                                      │   else  → throw exception │
                                      │ }                        │
                                      └──────────────────────────┘
```

```java
// From the controller's perspective — it calls AccountService directly
// In reality, it calls the proxy, which calls AccountService if authorized

@Autowired
private AccountService accountService;   // this is actually the proxy

accountService.getAccount("123");        // proxy intercepts this call
```

© 2026 by Innovation In Software Corporation

Method Security

- Self-invocation limitation
  - Anti-pattern to be avoided
  - this.getAccount() calls the real object, not the proxy
  - The proxy is only involved when an external caller invokes the method
  - AOP cannot intercept internal method calls within the same class

```java
@Service
public class AccountService {

    public void processAccounts() {
        // This calls getAccount() directly on 'this' — NOT through the proxy
        Account account = this.getAccount("123");  // @PreAuthorize NOT enforced
        // OR equivalently:
        Account account = getAccount("123");        // @PreAuthorize NOT enforced
    }

    @PreAuthorize("hasAuthority('SCOPE_read:accounts')")
    public Account getAccount(String accountId) {
        return accountRepository.findById(accountId);
    }
}
```

© 2026 by Innovation In Software Corporation

Method Security

- Self-invocation solution
  - The self-injection option 1
    - Requires the bean to be aware of its own proxy
    - Use it only when refactoring to a separate bean is not practical
  - The practical recommendation option 2
    - Structure your service layer so that security-sensitive methods are always called from outside the class
    - If a method is only ever called internally, it may not need @PreAuthorize
    - Or it needs to be on a separate bean.

```java
// Option 1: Inject the bean into itself (Spring supports this)
@Autowired
private AccountService self;
self.getAccount("123");  // goes through the proxy

// Option 2: Move the method to a separate bean
accountHelper.getAccount("123");  // external call → proxy involved

// Option 3: Apply security at the caller level instead
```

© 2026 by Innovation In Software Corporation

Method Security

- Method security is disabled by default
- What @EnableMethodSecurity activates
  - Creates AOP advisors that intercept annotated method calls
  - Enables
    - @PreAuthorize and @PostAuthorize
    - @PreFilter and @PostFilter
    - Enables @Secured (disabled by default, opt-in via securedEnabled = true)
  - Enables JSR-250 annotations like @RolesAllowed

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity          // ← activates AOP proxy infrastructure
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Ex
        // ... filter chain configuration
    }
}
```

© 2026 by Innovation In Software Corporation

Method Security

- @PreAuthorize
  - Evaluated before the method executes
  - If the expression evaluates to false
    - AccessDeniedException is thrown
    - The method body does not execute
    - No database queries run
    - No side effects occur

```java
@Service
public class AccountService {

    // Requires a specific OAuth2 scope
    @PreAuthorize("hasAuthority('SCOPE_read:accounts')")
    public List<Account> getAllAccounts() { ... }

    // Requires a role
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteAllAccounts() { ... }

    // Requires either a scope OR a role
    @PreAuthorize("hasAuthority('SCOPE_read:accounts') || hasRole('ADMIN')")
    public Account getAccount(String accountId) { ... }

    // Requires authentication (any authenticated user)
    @PreAuthorize("isAuthenticated()")
    public UserProfile getCurrentProfile() { ... }

    // References a method argument — the # prefix
    @PreAuthorize("hasAuthority('SCOPE_write:accounts') && #request.ownerId == authentication.name")
    public Account createAccount(CreateAccountRequest request) { ... }
}
```

© 2026 by Innovation In Software Corporation

Method Security

- @PostAuthorize
  - Authorization on the return value
  - Evaluated after the method executes
    - The return value is available
- When to use @PostAuthorize
  - The authorization decision depends on data that must be loaded from the database
  - Ownership checks where the owner ID is stored in the resource itself
  - Cases where the resource ID alone is not enough to make the authorization decision

```java
// Load the account first, then verify the caller owns it
@PostAuthorize("returnObject.ownerId == authentication.name || hasRole('ADMIN')")
public Account getAccount(String accountId) {
    return accountRepository.findById(accountId);  // runs before the check
}
```

@PreAuthorize	@PostAuthorize
Checked BEFORE method runs	Checked AFTER method runs
Method does not execute if denied	Method ALWAYS executes
No side effects on denial	Side effects may have occurred
More efficient	Required when return value needed

© 2026 by Innovation In Software Corporation

Method Security

- @PreFilter @PostFilter
  - Filtering collections based on per-element security expressions
  - @PreFilter filters the input collection before the method runs
  - @PostFilter filters the return collection after the method runs
    - filterObject: refers to each element in the collection

```
// Remove any accounts from the input that the caller doesn't own
@PreFilter("filterObject.ownerId == authentication.name")
public void updateAccounts(List<Account> accounts) {
    // 'accounts' only contains accounts owned by the caller
    accountRepository.saveAll(accounts);
}
```

```
// Return only accounts owned by the caller (or all if admin)
@PostFilter("filterObject.ownerId == authentication.name || hasRole('ADMIN')")
public List<Account> getAccounts() {
    return accountRepository.findAll();  // loads all, then filters
}
```

© 2026 by Innovation In Software Corporation

Method Security

- When to use these
  - Fine-grained data filtering without changing the service method logic
  - Returning a subset of results based on ownership
  - Filtering bulk operation inputs to only permitted items
- The trade-off with @PostFilter
  - The full collection is loaded from the database before filtering
  - For large datasets, this is inefficient

```java
// Remove any accounts from the input that the caller doesn't own
@PreFilter("filterObject.ownerId == authentication.name")
public void updateAccounts(List<Account> accounts) {
    // 'accounts' only contains accounts owned by the caller
    accountRepository.saveAll(accounts);
}
```

```java
// Return only accounts owned by the caller (or all if admin)
@PostFilter("filterObject.ownerId == authentication.name || hasRole('ADMIN')")
public List<Account> getAccounts() {
    return accountRepository.findAll();  // loads all, then filters
}
```

© 2026 by Innovation In Software Corporation

JSR-250 Annotations

- Older annotation styles
  - Encountered in existing enterprise codebases
  - @Secured
    - Spring's original method security annotation
  - @RolesAllowed
    - Java standard annotation

```java
// Simple role check — no SpEL, less expressive than @PreAuthorize
@Secured("ROLE_ADMIN")
public void deleteAccount(String accountId) { ... }

@Secured({"ROLE_ADMIN", "ROLE_MANAGER"})  // OR logic
public List<Account> getAccounts() { ... }
```

```java
@RolesAllowed("ADMIN")           // note: no ROLE_ prefix needed
public void deleteAccount(String accountId) { ... }

@RolesAllowed({"ADMIN", "MANAGER"})
public List<Account> getAccounts() { ... }
```

Annotation	SpEL Support	Standard	Expressive
`@PreAuthorize`	✅ Full	Spring	✅ Highest
`@Secured`	❌ None	Spring	Limited
`@RolesAllowed`	❌ None	JSR-250	Limited

© 2026 by Innovation In Software Corporation

Security Exceptions

- When @PreAuthorize denies access
  - Spring Security handles it in HTTP contexts
    - Do not catch AccessDeniedException in your service code
    - Let it propagate to the ExceptionTranslationFilter or @RestControllerAdvice
  - The non-HTTP context
    - When a service method with @PreAuthorize is called from a service method
    - There is no HTTP response to write a 403 to
    - The AccessDeniedException will propagate up the call stack and must be handled by the caller
    - Should be routed to an error handler

```
Caller invokes service method
        |
        ▼
AOP Proxy evaluates @PreAuthorize
Expression is false
        |
        ▼
throws AccessDeniedException
        |
        ├── Called from HTTP context?
        |     |
        |     ▼
        |   ExceptionTranslationFilter catches it
        |   AccessDeniedHandler produces 403 response
        |
        └── Called from non-HTTP context (Kafka, batch)?
              |
              ▼
            Exception propagates to the caller
            Caller must handle it explicitly
```

© 2026 by Innovation In Software Corporation

Security Exceptions

- When @PreAuthorize denies access
  - @RestControllerAdvice
  - Preferred over catching AccessDeniedException in individual methods
    - It provides consistent error handling across the entire API surface
    - Keeps service methods clean of exception handling concerns.

```java
@RestControllerAdvice
public class SecurityExceptionHandler {

    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public ErrorResponse handleAccessDenied(AccessDeniedException ex) {
        return new ErrorResponse(
            "access_denied",
            "You do not have permission to perform this operation"
        );
    }
}
```

© 2026 by Innovation In Software Corporation

Layered Security

- A practical heuristic:
  - If the rule can be expressed as a URL pattern, consider the web layer
  - If the rule involves method arguments, the return value, or a non-HTTP entry point, use the method layer
  - For critical operations (anything that modifies data or accesses sensitive information), use both

Authorization Rule	Web Layer	Method Layer	Recommendation
Public vs protected endpoints	✅	⚠️	Web layer
All endpoints require authentication	✅	⚠️	Web layer (`anyRequest().authenticated()`)
Broad role for an entire area (`/admin/**`)	✅	✅	Web layer — simpler
Specific scope per HTTP operation	✅	✅	Either — method layer is more explicit
Ownership check (your own resource)	❌	✅	Method layer only
Service called from Kafka/batch	❌	✅	Method layer only
Defence in depth for critical operations	✅	✅	Both

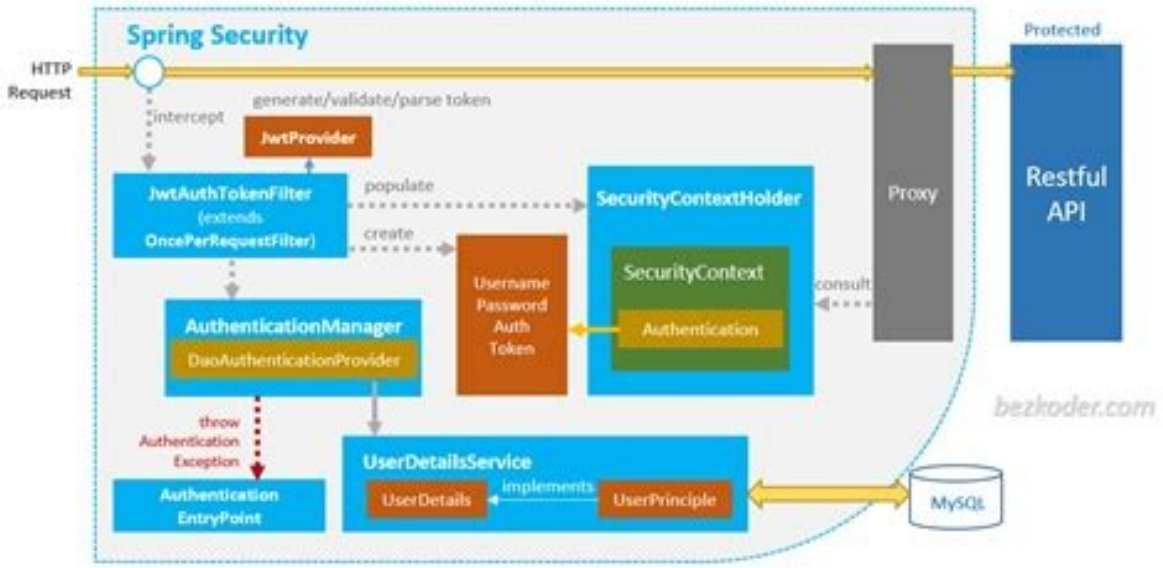
© 2026 by Innovation In Software Corporation

Testing

© 2026 by Innovation In Software Corporation

Testing

- Security configuration is behavior
  - Requires functional testing
    - A misconfigured permitAll() is a security vulnerability
    - A missing @PreAuthorize is a security vulnerability
  - Neither shows up as a compilation error or runtime exception under normal operation
  - What must be tested
    - Protected endpoints return 401 for unauthenticated requests
    - Protected endpoints return 403 for authenticated but unauthorized requests
    - Authorized requests succeed and reach the application
    - Method-level @PreAuthorize rules are enforced
    - Public endpoints are accessible without authentication

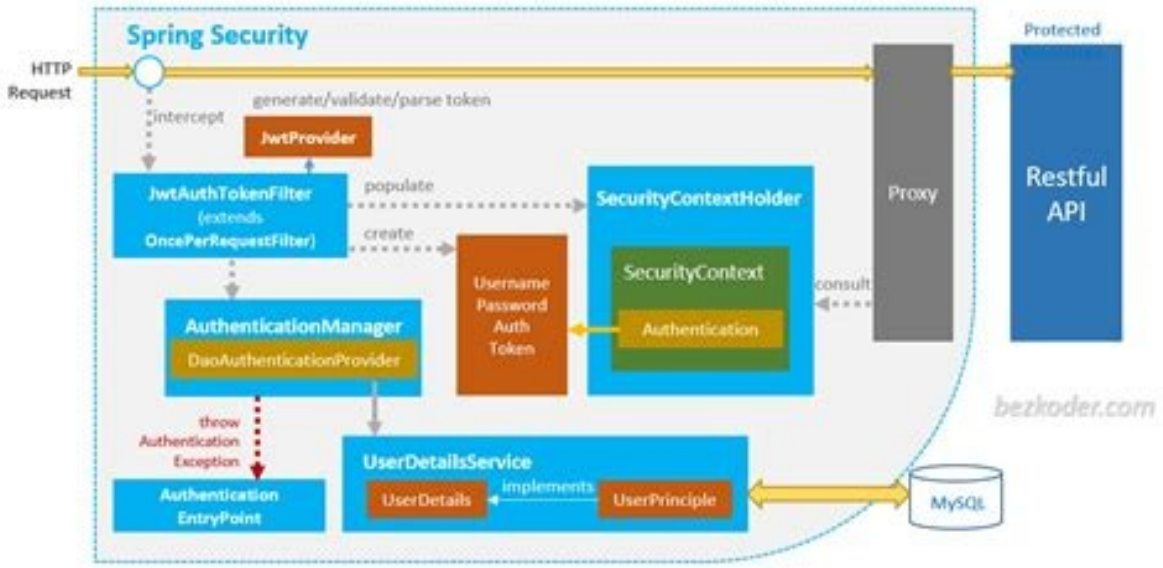


© 2026 by Innovation In Software Corporation

Image Credit: https://www.ppgbbe.intranet.biologia.ufrj.br/prueba-de-orina-que-tan-efectiva-es/prueba-de-orina-que-tan-efectiva-es/spring-security-authentication-and-authorization-example-u-jdevM3Cl

Testing

- Security rules are not self-testing
  - A missing @PreAuthorize on a method
    - Will never be caught by the compiler
    - Or by a test that only checks the happy path of authorized user accessing the resource
    - You must explicitly write tests that verify unauthorized access is denied
  - Security rules tend to change over time
    - New endpoints are added, existing rules are modified
    - Without tests, these changes can silently remove protections
  - Every @PreAuthorize annotation should have at least two tests
    - One that verifies the authorized case succeeds, and one that verifies the unauthorized case is denied

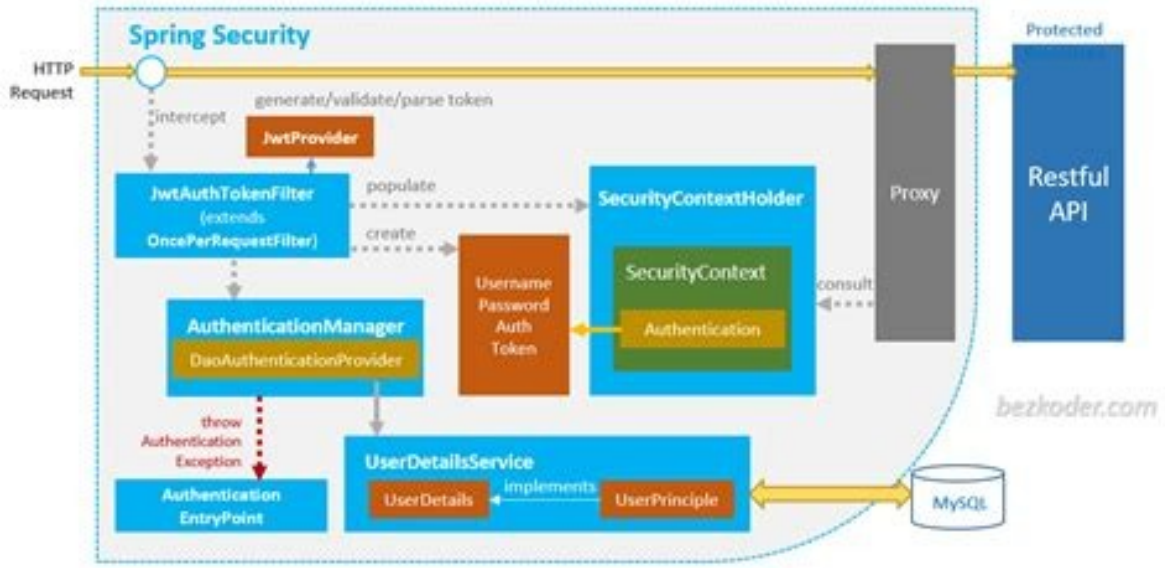


© 2026 by Innovation In Software Corporation

Image Credit: https://www.ppgbbe.intranet.biologia.ufrj.br/prueba-de-orina-que-tan-efectiva-es/prueba-de-orina-que-tan-efectiva-es/spring-security-authentication-and-authorization-example-u-jdevM3Cl

Testing

- Spring Security's testing support
  - @WithMockUser
    - Simulates an authenticated user without real tokens
  - @WithAnonymousUser
    - Simulates an unauthenticated request
  - SecurityMockMvcRequestPostProcessors
    - Inject mock authentication into MockMvc requests
  - @WithSecurityContext
    - custom security contexts for complex scenarios



© 2026 by Innovation In Software Corporation

Image Credit: https://www.ppgbbe.intranet.biologia.ufrj.br/prueba-de-orina-que-tan-efectiva-es/prueba-de-orina-que-tan-efectiva-es/spring-security-authentication-and-authorization-example-u-jdevM3Cl

Testing

- @WithMockUser
  - Populates the SecurityContext with a mock Authentication object
    - No real JWT is needed
    - The mock user has the specified username, roles, and authorities.
    - The filter chain runs normally, but the authentication step is bypassed because the SecurityContext is already populated
  - The four tests shown represent the minimum security test suite for a protected endpoint:
    - Authorized access succeeds
    - Unauthenticated access returns 401
    - Authenticated but unauthorized access returns 403
    - Admin access succeeds (if there is a role hierarchy)

```java
@SpringBootTest
@AutoConfigureMockMvc
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    @WithMockUser(username = "alice", roles = {"USER"})
    void getAccount_authenticatedUser_returns200() throws Exception {
        mockMvc.perform(get("/accounts/123"))
            .andExpect(status().isOk());
    }

    @Test
    void getAccount_unauthenticated_returns401() throws Exception {
        mockMvc.perform(get("/accounts/123"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(username = "alice", roles = {"USER"})
    void deleteAccount_insufficientRole_returns403() throws Exception
        mockMvc.perform(delete("/accounts/123"))
            .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(username = "alice", roles = {"ADMIN"})
    void deleteAccount_adminRole_returns200() throws Exception {
        mockMvc.perform(delete("/accounts/123"))
            .andExpect(status().isOk());
    }
}
```

© 2026 by Innovation In Software Corporation

Testing

- @WithMockUser
  - Testing OAuth2 scopes and JWT claims

```java
@Test
void getAccount_withReadScope_returns200() throws Exception {
    mockMvc.perform(get("/accounts/123")
        .with(jwt()
            .jwt(token -> token
                .subject("alice")
                .claim("scope", "read:accounts write:accounts")
            )
            .authorities(
                new SimpleGrantedAuthority("SCOPE_read:accounts"),
                new SimpleGrantedAuthority("SCOPE_write:accounts")
            )
        )
    )
    .andExpect(status().isOk());
}

@Test
void getAccount_withoutReadScope_returns403() throws Exception {
    mockMvc.perform(get("/accounts/123")
        .with(jwt()
            .authorities(new SimpleGrantedAuthority("SCOPE_write:accounts"))
        )
    )
    .andExpect(status().isForbidden());
}
```

© 2026 by Innovation In Software Corporation

Testing

- Testing method security
  - Testing @PreAuthorize in isolation from the web layer
  - Testing method security at the service layer is important
    - because it verifies the @PreAuthorize annotations are actually being enforced by the AOP proxy
    - A service test that uses @SpringBootTest will have the full AOP infrastructure active
  - However
    - @WebMvcTest does not load the full application context and does not activate method security AOP
    - For testing @PreAuthorize annotations, you need @SpringBootTest or a custom test configuration that explicitly enables method security

```java
@SpringBootTest
class AccountServiceTest {

    @Autowired
    private AccountService accountService;

    @Test
    @WithMockUser(authorities = {"SCOPE_read:accounts"})
    void getAccount_withReadAuthority_succeeds() {
        assertDoesNotThrow(() ->
            accountService.getAccount("123")
        );
    }

    @Test
    @WithMockUser(authorities = {"SCOPE_write:accounts"})
    void getAccount_withoutReadAuthority_throwsAccessDenied() {
        assertThrows(AccessDeniedException.class, () ->
            accountService.getAccount("123")
        );
    }

    @Test
    void getAccount_unauthenticated_throwsAuthenticationException() {
        // No @WithMockUser — SecurityContext is empty
        assertThrows(AuthenticationException.class, () ->
            accountService.getAccount("123")
        );
    }
}
```

© 2026 by Innovation In Software Corporation

Testing

- Testing method security
  - A common pattern is to have both layers of tests
    - @WebMvcTest for testing the web layer security (URL rules, 401/403 responses)
    - @SpringBootTest for testing method security (service-layer rules)
  - This separation keeps tests fast
    - Web layer tests don't load the full context while still verifying method security
- We will return to testing in depth later in the course

```java
@SpringBootTest
class AccountServiceTest {

    @Autowired
    private AccountService accountService;

    @Test
    @WithMockUser(authorities = {"SCOPE_read:accounts"})
    void getAccount_withReadAuthority_succeeds() {
        assertDoesNotThrow(() ->
            accountService.getAccount("123")
        );
    }

    @Test
    @WithMockUser(authorities = {"SCOPE_write:accounts"})
    void getAccount_withoutReadAuthority_throwsAccessDenied() {
        assertThrows(AccessDeniedException.class, () ->
            accountService.getAccount("123")
        );
    }

    @Test
    void getAccount_unauthenticated_throwsAuthenticationException() {
        // No @WithMockUser — SecurityContext is empty
        assertThrows(AuthenticationException.class, () ->
            accountService.getAccount("123")
        );
    }
}
```

© 2026 by Innovation In Software Corporation

Recap

© 2026 by Innovation In Software Corporation

```
HTTP Request
      |
      ▼
┌──────────────────────────────────────────────────────────────┐
│                      LAYER 1: FILTER CHAIN                   │
│                                                              │
│  Extract token → Verify JWT → Populate SecurityContext       │
│                                                              │
│  URL Authorization: permitAll / authenticated / hasRole      │
│  Exception Handling: 401 AuthenticationEntryPoint            │
│                      403 AccessDeniedHandler                 │
│  Security Headers: X-Frame-Options, HSTS, Cache-Control ...  │
└──────────────────────────────────────────────────────────────┘
                            | Authenticated & URL rule satisfied
                            ▼
                 Controller → Service Method
                            |
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                      LAYER 2: AOP PROXY                      │
│                                                              │
│  @PreAuthorize — evaluated before method runs                │
│  @PostAuthorize — evaluated against return value             │
│  @PreFilter / @PostFilter — filter collection arguments/results │
│                                                              │
│  Reads the same SecurityContext populated by Layer 1         │
│  Works for HTTP, Kafka consumers, batch jobs, all entry points │
└──────────────────────────────────────────────────────────────┘
                            | Method security satisfied
                            ▼
                  Your Service Logic Runs
```

© 2026 by Innovation In Software Corporation

Questions?